

Razvoj web aplikacija - Predavanje 7

Frontend: arhitektura SPA, router i layout

mag.ing. Josip Torić (SIT UNIZD)

23.11.2026.



Studij informacijske tehnologije
Sveučilišta u Zadru

Plan predavanja

- 1 Što je SPA i zašto ne radimo MPA
- 2 Vue 3 i Composition API
- 3 Vite: dev server i build pipeline
- 4 Postavljanje projekta (scaffold)
- 5 Vue Router: rute, parametri, nested rute
- 6 Layout i navigacija
- 7 Stilizacija: scoped styles i globalni CSS
- 8 TypeScript: setup i integracija s Vue 3
- 9 Commitovi za ovo predavanje

Na kraju predavanja: Vue projekt s routingom, layoutom i stubiranim stranicama za admin i club korisnika.

Ishodi učenja

Nakon ovog predavanja student može:

- objasniti razliku između SPA i klasične server-rendered aplikacije
- kreirati Vue 3 projekt s Vite-om i TypeScriptom
- definirati rute s parametrima i nested strukturom u Vue Routeru
- implementirati zajednički layout s navigacijom
- organizirati frontend projekt po mapama (views, components, router, stores, api)
- tipizirati props, Pinia store i Axios pozive TypeScriptom

SPA vs MPA

MPA (Multi-Page Application)

- Server generira HTML za svaku stranicu
- Svaki klik = novi HTTP request + full page reload
- Backend i frontend su spojeni (Django templates, Jinja2, PHP)
- Jednostavniji SEO

SPA (Single-Page Application)

- Browser učitava jedan HTML + JS bundle
- Navigacija mijenja DOM bez reloada
- Frontend i backend komuniciraju kroz API (JSON)
- Brži UX nakon inicijalnog loada

Mi radimo SPA jer već imamo REST API. Frontend je potpuno odvojen od backenda - može ga razvijati drugi tim.

Kako SPA zapravo radi

- 1 Browser dohvati `index.html` - jedini HTML koji server ikad pošalje
- 2 HTML učitava JavaScript bundle - entry point je `src/main.ts`
- 3 JS preuzima renderiranje: kreira DOM, sluša eventove, mijenja prikaz
- 4 Klik na link ne šalje request serveru - JS mijenja URL i renderira novu komponentu
- 5 Podaci dolaze asinkrono kroz API pozive (JSON)

Server nakon inicijalnog loada služi samo kao API. Cijeli UI živi u browseru.

Prednosti odvojenog frontenda

Prednost	Objašnjenje
Nezavisni deploy	Frontend i backend se deployaju zasebno. Promjena boje gumba ne zahtijeva deploy backenda.
Različiti timovi	Frontend i backend tim rade paralelno. Dogovor je API ugovor (Swagger).
API-first	Isti backend služi web, mobilnu aplikaciju i integracije.
Tehnološka sloboda	Frontend može biti Vue, React ili mobilna app - backend ne zna i ne mari.

Reaktivni sustav: koncept

Vue koristi **reaktivni sustav** koji automatski prati zavisnosti između podataka i UI-ja.

podatak se promijeni → Vue detektira → DOM se ažurira

- Nema ručnog `document.getElementById().innerHTML = ...`
- Vue interno koristi JavaScript Proxy za praćenje pristupa i promjena
- Kad čitaš varijablu u templateu, Vue zabilježi tu zavisnost
- Kad se varijabla promijeni, Vue zna točno koje dijelove DOM-a treba ažurirati

Ovo je ključna razlika od jQuery/vanilla JS pristupa - ne razmišljate o DOM-u, razmišljate o podacima.

Virtual DOM

Vue ne mijenja stvarni DOM direktno. Koristi **Virtual DOM** - laganu JavaScript reprezentaciju UI-ja.

- 1 Komponenta renderira Virtual DOM stablo (JS objekti)
- 2 Kad se podatak promijeni, renderira se novo Virtual DOM stablo
- 3 Vue uspoređuje staro i novo stablo (diffing)
- 4 Samo razlike se primijene na stvarni DOM

Stvarni DOM je spor - svaka promjena može triggerirati layout recalculation, repaint i reflow. Virtual DOM minimizira broj stvarnih DOM operacija.

Vue 3: zašto baš Vue

- Manji learning curve od Reacta - template syntax je bliži HTML-u
- Composition API daje fleksibilnost sličnu React Hooksima
- Ekosustav: Vue Router, Pinia, Vite - sve službeno i dobro integrirano
- Odlična dokumentacija na vuejs.org

Vue koristi **reaktivni sustav** - kad se podatak promijeni, UI se automatski ažurira. Nema ručnog DOM manipuliranja.

Composition API vs Options API

Vue 3 nudi dva načina pisanja komponenti. Mi koristimo **Composition API** jer je fleksibilniji i bolje skalira.

Options API

- `data()`, `methods`, `computed`, `watch`
- Logika raspoređena po sekcijama
- Teško za ponovnu upotrebu logike

Composition API

- `ref()`, `computed()`, `watch()`
- Logika grupirana po značenju
- Composables za dijeljenje logike
- `<script setup>` sintaksa

ref() vs reactive()

	ref()	reactive()
Tip	Primitivi i objekti	Samo objekti
Pristup	<code>count.value</code> u scriptu, <code>count</code> u templateu	Direktno: <code>state.count</code>
Reassign	Da (<code>count.value = 5</code>)	Ne (gubi reaktivnost)
Destruktuiranje	Sigurno s <code>toRefs()</code>	Gubi reaktivnost

Preporuka: koristite `ref()` za sve. Jednostavniji je, radi s primitivima i objektima, i `.value` čini eksplicitnim da radite s reaktivnom varijablom.

Životni ciklus komponente

Svaka Vue komponenta prolazi kroz definirane faze:

kreiranje → montiranje → ažuriranje → uništavanje

Hook	Kad se poziva
onMounted	Komponenta je renderirana u DOM. API pozivi idu ovdje.
onUpdated	Reaktivni podatak se promijenio, DOM je ažuriran.
onUnmounted	Komponenta je uklonjena. Cleanup (intervali, listeneri).
onBeforeUnmount	Neposredno prije uklanjanja.

onMounted ćete koristiti najčešće - za dohvat podataka kad se stranica učita.

Template sintaksa: direktive

Vue direktive su specijalni atributi koji počinju s `v-`:

Direktiva	Značenje
<code>v-if / v-else</code>	Uvjetno renderiranje - element ne postoji u DOM-u
<code>v-show</code>	Uvjetno sakrivanje - element postoji ali je <code>display: none</code>
<code>v-for</code>	Iteracija kroz array ili objekt
<code>v-bind (:)</code>	Dinamički atribut - <code>:class</code> , <code>:href</code> , <code>:disabled</code>
<code>v-on (@)</code>	Event listener - <code>@click</code> , <code>@submit</code> , <code>@input</code>
<code>v-model</code>	Dvosmjerni binding - spaja input s varijablom

`v-if` uklanja element iz DOM-a (skuplji toggle, jeftiniji init). `v-show` ga samo skriva (jeftiniji toggle, skuplji init).

Props i Emits

Parent-child komunikacija u Vueu je **jednosmjerna**:

Parent $\xrightarrow{\text{props}}$ Child $\xrightarrow{\text{emit}}$ Parent

- **Props** - podaci teku od parenta prema childu. Child ih nikad ne smije mijenjati.
- **Emits** - child šalje event parentu kad se nešto dogodi (klik, submit, promjena).
- Parent odlučuje što napraviti s eventom - child ne zna.

`defineProps()` deklarira koje propse komponenta prima. `defineEmits()` deklarira koje evente emitira. Oboje su compile-time makroi u `<script setup>`.

Computed vs Watch

computed()

- Derivirani podatak iz postojećih
- Kešira se - recalculira samo kad se zavisnosti promijene
- Read-only (uglavnom)
- Primjer: filtrirani popis, ukupna cijena

watch()

- Reagira na promjenu podatka
- Pokreće side-effect (API poziv, log)
- Ne kešira ništa
- Primjer: dohvat podataka kad se ruta promijeni

Ako možeš izraziti nešto kao `computed`, koristi `computed`. `watch` je za situacije kad promjena podatka treba pokrenuti akciju van renderiranja.

Slots: kompozicija komponenti

Slots omogućuju parentu da „ubaci” sadržaj unutar child komponente:

Tip slota	Upotreba
Default slot	Jedan blok sadržaja - <code><MyCard>tekst</MyCard></code>
Named slots	Više blokova - <code><template #header></code> , <code><template #footer></code>
Scoped slots	Child pruža podatke parentu za renderiranje

Slots su Vue verzija „composition over inheritance”. Umjesto da child komponenta zna sve o sadržaju koji prikazuje, parent šalje sadržaj, a child ga pozicionira.

Prva Vue komponenta

```
<script setup lang="ts">
import { ref, computed } from 'vue'

const count = ref(0)
const doubled = computed(() => count.value * 2)

function increment() {
  count.value++
}
</script>

<template>
  <button @click="increment">
    Klikova: {{ count }} (duplo: {{ doubled }})
  </button>
</template>
```

ref() stvara reaktivnu varijablu. U scriptu se pristupa s .value, u templateu direktno.

Scaffold projekta s TypeScriptom

```
# Interaktivni wizard - odaberi TypeScript: Yes
npm create vue@latest web

cd web
npm install

# Dodaj preostale dependencije
npm install axios

# Type-check bez bundlanja
npm run type-check # vue-tsc --noEmit

# Dev server s hot-reload
npm run dev # http://localhost:5173
```

Wizard konfigurira tsconfig.json, env.d.ts i Vue Router + Pinia po izboru. Sve .vue datoteke automatski dobivaju `<script setup lang="ts">`.

TypeScript u Vue projektu: zašto

- **Greške pri pisanju, ne pri izvršavanju** - TypeScript javlja pogrešan tip prije nego što browser uopće pokrene kod
- **Autocomplete koji zna kontekst** - IDE prikazuje dostupna polja na API responseu, propse komponente, metode storea
- **Sigurni refactoring** - preimenuješ interface, TypeScript pokaže svako mjesto koje treba popraviti
- **Vue 3 je pisan u TypeScriptu** - podrška je ugrađena od početka, nema workaroundova ni zastarjelih tipova

TypeScript ne usporava razvoj na ovom projektu - ubrzava ga jer eliminira cijelu kategoriju runtime bugova (undefined is not a function, pogrešan naziv polja).

Tipiziranje komponenti: defineProps i defineEmits

```
// src/components/FormField.vue
<script setup lang="ts">

// Genericki interface - bez runtime overhead
defineProps<{
  label: string
  error?: string
  modelValue: string | number
}>()

defineEmits<{
  'update:modelValue': [value: string]
}>()

// Domain objekt - dijeli se s Pinia storeom
interface Lifter {
  id: number
  first_name: string
  last_name: string
  gender: 'M' | 'F'
}
</script>
```

Tipiziranje Pinia storea

```
// src/stores/lifters.ts
import { defineStore } from 'pinia'
import { ref } from 'vue'
import type { Lifter } from '@types'

export const useLifterStore = defineStore('lifters', () => {
  const lifters = ref<Lifter[]>([])
  const loading = ref(false)
  const error = ref<string | null>(null)

  async function fetchLifters(clubId: number): Promise<void> {
    loading.value = true
    // ...
  }

  return { lifters, loading, error, fetchLifters }
})
```

`useLifterStore()` vraća potpuno tipiziran objekt - autocomplete radi za cijeli store.

Tipiziranje Axios poziva

```
// src/api/lifters.ts
import api from './client'
import type { Lifter } from '@types'

export async function getLifters(clubId: number): Promise<Lifter[]> {
  const { data } = await api.get<Lifter[]>(
    `/clubs/${clubId}/lifters`
  )
  return data // TypeScript zna: data je Lifter[]
}

export async function createLifter(
  clubId: number,
  body: Omit<Lifter, 'id'>
): Promise<Lifter> {
  const { data } = await api.post<Lifter>(
    `/clubs/${clubId}/lifters`, body
  )
  return data
}
```

Centralni types/ folder

Interfacei se definiraju jednom, koriste svugdje:

File	Sadržaj
src/types/index.ts	Lifter, Club, Competition, Registration
src/types/auth.ts	User, TokenPair, LoginPayload
src/types/api.ts	PaginatedResponse<T>, ApiError

Preslikavanje s Pydantic schemom je 1:1 - isti nazivi polja, isti tipovi. Kad backend promijeni schemu, TypeScript u frontendu odmah pokaže gdje se koristi staro polje. Ovo je prednost API-first pristupa.

tsconfig: relevantne opcije

Opcija	Što radi
<code>strict: true</code>	Sve provjere uključene (preporučeno za novi projekt)
<code>paths: {"@/*": ["./src/*"]}</code>	Alias @/ za src/ - mora biti isti kao vite.config.ts
<code>include: ["src/**/*.ts"]</code>	TypeScript gleda samo naš kod, ne node_modules
<code>vue-tsc</code>	Type-check .vue datoteka - tsc sam ne razumije Vue SFC

`npm run type-check` pokreće `vue-tsc --noEmit` - provjerava tipove bez generiranja outputa. Dodajte ga u CI pipeline uz lint i testove.

Vite vs Webpack

	Webpack	Vite
Dev start	Bundla cijeli projekt prije starta (sporo)	Servira module nativno (instant)
HMR	Cijeli modul se zamijeni	Samo promijenjeni modul
Konfiguracija	Kompleksna (webpack.config.js)	Minimalna (vite.config.ts)
Prod build	Webpack bundler	Rollup bundler

Vite koristi činjenicu da moderni browseri podržavaju ES modules nativno. U developmentu nema bundlanja - svaki file se servira pojedinačno. Rezultat: dev server se pokrene za sekunde umjesto minuta.

Struktura frontend projekta

web/src/

- views/ - stranice (jedna po ruti)
- components/ - reusable UI dijelovi
- router/ - definicija ruta
- stores/ - Pinia storeovi
- api/ - HTTP client i pozivi
- App.vue - root komponenta
- main.ts - entry point

Konvencije:

- Views su stranice -
LoginView.vue,
DashboardView.vue
- Components su dijelovi -
NavBar.vue,
LoadingSpinner.vue
- Jedan view = jedna ruta
- Store po domeni - authStore,
lifterStore

Vue Router: osnovna konfiguracija

```
// src/router/index.ts
import { createRouter, createWebHistory } from 'vue-router'
import LoginView from '@views/LoginView.vue'
import DashboardView from '@views/DashboardView.vue'

const routes = [
  { path: '/login', name: 'login', component: LoginView },
  {
    path: '/',
    name: 'dashboard',
    component: DashboardView,
  },
]

const router = createRouter({
  history: createWebHistory(),
  routes,
})

export default router
```

Rute s parametrima i nested rute

```
// Ruta s parametrom
{ path: '/clubs/:clubId/lifters', name: 'lifters',
  component: LiftersView }

// Pristup parametru u komponenti
const route = useRoute()
const clubId = route.params.clubId

// Nested rute - dijele layout
{
  path: '/admin',
  component: AdminLayout,
  children: [
    { path: 'clubs', component: AdminClubsView },
    { path: 'competitions', component: AdminCompetitionsView },
  ],
}
```

Nested rute koriste `<RouterView />` unutar parent komponente za renderiranje child rute.

Hash mode vs History mode

Vue Router podržava dva načina upravljanja URL-ovima:

	Hash mode	History mode
URL	<code>app.com/#/clubs</code>	<code>app.com/clubs</code>
Kako radi	Browser ignorira <code>#</code> dio	HTML5 History API
Server config	Ne treba	Treba fallback na <code>index.html</code>
SEO	Lošije (<code>#</code> se ne indexira)	Bolje

Mi koristimo `createWebHistory()` - history mode. URL-ovi izgledaju prirodno, ali server mora sve nepoznate rute preusmjeriti na `index.html` jer je routing na klijentu.

Lazy loading ruta

Kad aplikacija naraste, ne želimo učitati sav JavaScript odjednom:

- **Eager loading:** sve komponente se učitaju pri startu - sporiji inicijalni load
- **Lazy loading:** komponenta se učitava tek kad korisnik navigira na tu rutu

Umjesto `import AdminView from '@views/AdminView.vue'` koristimo:
`component: () => import('@views/AdminView.vue')`

Vite automatski radi **code splitting** - svaki lazy-loaded view postaje zasebni JS chunk. Korisnik preuzima samo kod za stranicu koju gleda.

Layout s navigacijom

```
<!-- src/App.vue -->
<script setup lang="ts">
import NavBar from '@components/NavBar.vue'
</script>

<template>
  <NavBar />
  <main class="container">
    <RouterView />
  </main>
</template>
```

<RouterView /> je placeholder - Vue Router renderira komponentu koja odgovara trenutnoj ruti. Navigacija (NavBar) ostaje fiksna jer je izvan <RouterView />.

Stilizacija: scoped styles u SFC-u

Treći blok svake .vue datoteke je <style>. S atributom scoped stilovi vrijede samo za tu komponentu:

```
<!-- src/components/NavBar.vue -->
<style scoped>
nav {
  display: flex;
  gap: 1rem;
  padding: 0.75rem 1rem;
  border-bottom: 1px solid #ddd;
}
.error { color: #b00020; font-size: 0.875rem; }
button:disabled { opacity: 0.5; cursor: not-allowed; }
</style>
```

Vue svakoj komponenti doda jedinstveni data-atribut i prepíše selektore - zato .error iz NavBar-a ne curi u formu. Svaka klasa koju koristite u templateu (error, badge-green, btn-danger) mora biti definirana ovdje ili globalno.

Globalni stilovi i responsivnost

```
/* src/assets/main.css - importan u main.ts */
:root { --primary: #1f6feb; }
.container { max-width: 960px;
             margin: 0 auto; padding: 1rem; }
.badge-green { background: #d1f7d6;
               padding: 2px 8px; border-radius: 8px; }

@media (max-width: 600px) {
  .container { padding: 0.5rem; }
  table { display: block; overflow-x: auto; }
}
```

Globalno ide ono što dijele sve komponente: reset, layout klase, badgevi. Scaffold već stvara assets/main.css - proširujete njega. @media breakpoint čini tablice upotrebljivima na mobitelu.

Za projekt je dovoljan vlastiti mali CSS. Komponentne biblioteke (PrimeVue, Vuetify) smijete koristiti, ali ih ne obrađujemo - na obrani morate objasniti i njihov kod.

Navigacija: RouterLink

```
<!-- src/components/NavBar.vue -->
<template>
  <nav>
    <RouterLink to="/">Dashboard</RouterLink>
    <RouterLink to="/clubs">Klubovi</RouterLink>
    <RouterLink :to="`/clubs/${clubId}/lifters`">
      Natjecatelji
    </RouterLink>
    <button @click="logout">Odjava</button>
  </nav>
</template>
```

RouterLink renderira `<a>` tag ali presreće klik - umjesto full page reloada, mijenja samo komponentu u `<RouterView />`.

Programska navigacija

```
import { useRouter } from 'vue-router'

const router = useRouter()

// Navigacija po imenu rute
router.push({ name: 'lifters', params: { clubId: 5 } })

// Navigacija po pathu
router.push('/login')

// Zamjena (ne dodaje u history)
router.replace('/dashboard')

// Natrag
router.back()
```

`router.push` dodaje u browser history (back button radi). `router.replace` zamjenjuje trenutni entry - koristi se nakon logina da korisnik ne može ići „back” na login formu.

Konvencije i alati

Konvencije imenovanja:

Što	Konvencija	Primjer
Komponenta	PascalCase	NavBar.vue, LifterForm.vue
Props / emit	camelCase / kebab-case	firstName, @item-click
Composables	use prefix	useAuth(), useLifters()
Store	use...Store	useAuthStore()

Vue DevTools - browser ekstenzija, obavezni alat za razvoj:

- **Components** - stablo komponenti, props i lokalni state
- **Pinia** - svi storeovi, njihov state, mogućnost uređivanja
- **Routes** - trenutna ruta, parametri, matched records

Kad nešto ne radi - otvorite Vue DevTools. Vidjet ćete je li komponenta renderirana, imaju li propsi ispravne vrijednosti i je li store ažuriran.

Česte greške početnika u Vueu

- ❶ **Mutiranje propsa** - child mijenja prop direktno. Vue javlja warning, ali podatak se ne ažurira u parentu. Koristiti emit.
- ❷ **Zaboravljen :key u v-for** - bez keya Vue ne može efikasno pratiti elemente. Koristiti `:key="item.id"`, nikad index.
- ❸ **Reaktivnost na nested objektima** - `ref(a: b: 1)` - promjena `obj.value.a.b` se detektira jer Vue 3 koristi Proxy, ali zamjena cijelog objekta zahtijeva `.value = {...}`.
- ❹ **Zaboravljen .value** - u `<script setup>` pristupate ref s `.value`, u templateu bez njega. Miješanje dovodi do bugova.

Registracija routera i Pinie u main.ts

```
// src/main.ts
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'
import router from './router'

const app = createApp(App)
app.use(createPinia())
app.use(router)
app.mount('#app')
```

`app.use()` registrira plugine. Redosljed je bitan - Pinia prije routera jer route guardovi mogu koristiti storeove.

Commitovi za Predavanje 7

- `feat(web)`: scaffold app layout, router, and navigation shell
- `feat(web)`: add route structure for admin and club areas
- `chore(web)`: add eslint/prettier and basic project conventions
- `docs`: add frontend structure and component guidelines

Što ide sljedeće (Predavanje 8)

- API komunikacija s backendom (Axios/fetch)
- Auth flow: login forma, token storage, refresh
- Route guardovi za zaštitu ruta po roli
- Error handling na frontendu

Pitanja?